

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #13: Query Execution II

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Background

Our previous discussion of query execution did not specify how the DBMS would compute results. We are now going to discuss how to organize the system's internals on the CPU.

The main goal is to understand how to enable the DBMS to support *parallel* query execution. This provides several benefits:

- Increased performance in throughput and latency.
- Increased availability.
- Potentially lower *total cost of ownership* (TCO). This cost includes both the hardware procurement and software license, as well as the labor overhead of deploying the DBMS and the energy needed to run the machines.

The techniques discussed here are applicable to distributed DBMSs as well, so we first need to understand the differences between a *parallel* and *distributed* DBMS. These systems spread the database out across multiple “resources” to improve parallelism. These resources are either computational (e.g., CPU cores, CPU sockets, GPUs, additional machines) or storage (e.g., disks, memory).

Parallel DBMS:

- Nodes are physically close to each other.
- Nodes are connected with high-speed LAN.
- Communication cost between nodes is assumed to be fast and reliable.

Distributed DBMS:

- Nodes can be far from each other.
- Nodes are connected using public network.
- Communication costs between nodes is slower and failures cannot be ignored.

Even though the database may be physically divided over multiple resources, it still appears as a single logical database instance to the application. Thus, the SQL query for a single-node DBMS should generate the same result on a parallel or distributed DBMS.

Types of Parallelism

- **Inter-Query:** The DBMS executes different queries concurrently. This increases throughput and reduces latency. Concurrency is tricky when queries are updating the database.
- **Intra-Query:** The DBMS executes the operations of a single query in parallel. This decreases latency for long-running queries.

2 Process Models

A DBMS *process model* defines how the system supports concurrent requests from a multi-user application/environment. The DBMS is comprised of more or more *workers* that are responsible for executing tasks on behalf of the client and returning the results.

Approach #1 – Process per Worker:

- Each worker is a separate OS process, and thus relies on OS scheduler.
- Use shared memory for global data structures.
- A process crash does not take down entire system.

Approach #2 – Process Pool:

- A worker uses any process that is free in a pool.
- Still relies on OS scheduler and shared memory.
- This approach can be bad for CPU cache locality due to no guarantee of using the same process between queries.

Approach #3 – Thread per Worker:

- Single process with multiple worker threads.
- DBMS has to manage its own scheduling.
- May or may not use a dispatcher thread.
- Although a thread crash (may) kill the entire system, we have to make sure that we write high-quality code to ensure that this does not happen.

Using a multi-threaded architecture has advantages that there is less overhead per context switch and you do not have to manage shared model. The thread per worker model does not mean that you have intra-query parallelism.

For each query plan, the DBMS has to decide where, when, and how to execute:

- How many tasks should it use?
- How many CPU cores should it use?
- What CPU core should the tasks execute on?
- Where should a task store its output?

3 Inter-Query Parallelism

The goal of this type of parallelism is to improve the DBMS's overall performance by allowing multiple queries to execute simultaneously.

We will cover this in more detail when we discuss concurrency control protocols.

4 Intra-Query Parallelism

The goal of this type of parallelism is to improve the performance of a single query by executing its operators in parallel. There are parallel algorithms for every relational operator.

Intra-Operator Parallelism

The query plan's operators are decomposed into independent instances that perform the same function on different subsets of data.

The DBMS inserts an **exchange** operator into the query plan to coalesce results from children operators.

The exchange operator prevents the DBMS from executing operators above it in the plan until it receives all of the data from the children.

In general, there are three types of exchange operators:

- **Gather:** Combine the results from multiple workers into a single output stream. This is the most common type used in parallel DBMSs.
- **Repartition:** Reorganize multiple input streams across multiple output streams. This allows the DBMS take inputs that are partitioned one way and then redistribute them in another way.
- **Distribute:** Split a single input stream into multiple output streams.

Inter-Operator Parallelism

The DBMS overlaps operators in order to pipeline data from one stage to the next without materialization. This is sometimes called **pipelined parallelism**.

This approach is widely used in *stream processing systems*, systems that continually execute a query over a stream of input tuples.

Bushy Parallelism

Extension of inter-operator parallelism where workers execute multiple operators from different segments of a query plan at the same time.

The DBMS still uses exchange operators to combine intermediate results from these segments.

5 I/O Parallelism

Using additional processes/threads to execute queries in parallel will not improve performance if the disk is always the main bottleneck. Thus, we need a way to split the database up across multiple storage devices.

Multi-Disk Parallelism

Configure OS/hardware to store the DBMS's files across multiple storage devices. Can be done through storage appliances and RAID configuration. This is transparent to the DBMS. It cannot have workers operate on different devices because it is unaware of the underlying parallelism.

File-based Partitioning

Some DBMSs allow you to specify the disk location of each individual database. The buffer pool manager maps a page to a disk location. This is also easy to do at the file-system level if the DBMS stores each database in a separate directory. However, the log file might be shared.

Logical Partitioning

Split single logical table into disjoint physical segments that are stored/managed separately. Such partitioning is ideally transparent to the application. That is, the application should be able to access logical tables without caring how things are stored.

Vertical Partitioning:

- Store a table's attributes in a separate location (like a column store).
- Have to store tuple information to reconstruct the original record.

Horizontal Partitioning:

- Divide the tuples of a table into disjoint segments based on some partitioning keys.
- There are different ways to decide how to partition (e.g., hash, range, or predicate partitioning). The efficacy of each approach depends on the queries.

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]